MPMC — Module 2 · PART 3: 8086 INSTRUCTION SET, DIRECTIVES & PROGRAMMING — EXAM-READY NOTES

Topic: All 6 instruction groups · String instructions · Branch & loop instructions · Assembler directives & operators · Procedures, CALL/RET, macros · Modular programming · Example programs

Source: Module 2 lecture slides (ECE3004 – Microprocessors and Microcontrollers).

Module 2 is split into 3 files — use them together, nothing is left out:

- **Part 1** → `MPMC_Module2_Part1_Architecture.md` — generations, pins, BIU/EU, registers, flags, physical address, interrupts
- **Part 2** → `MPMC_Module2_Part2_Addressing_Modes.md` — all 12 addressing modes
- **Part 3 (this file)** → instruction set, assembler directives, procedures/macros, programs
- (All-in-one master copy: `MPMC_Module2_Exam_Notes.md`)

CONTENTS OF THIS FILE

1. The 6 types of instructions — overview & notation
2. Data transfer instructions
3. Arithmetic instructions
4. Logical instructions (including shifts and rotates)
5. String manipulation instructions
6. Processor control instructions
7. Control transfer instructions (jumps, branches, loops)
8. Assembler directives and assembler operators
9. Procedures, CALL/RET, macros and modular programming
10. Example programs
11. Most likely exam questions (instruction set)
12. Last-minute revision cram sheet (instruction set)

1. THE 6 TYPES OF INSTRUCTIONS — OVERVIEW

The 8086 supports 6 types of instructions:

1. **Data Transfer** Instructions
2. **Arithmetic** Instructions
3. **Logical** Instructions
4. **String Manipulation** Instructions
5. **Processor Control** Instructions
6. **Control Transfer** Instructions

Notation used in this file (same as the slides):

- ( ) denotes **contents of**
- ← denotes **transfer / assignment**
- `reg2` = **destination**, `reg1` = **source** (as written in the slides)
- `MA` = memory address, `MAE` = memory address in the Extra segment, `BA` = base address, `EA` = effective address
- d8 / d16 = 8-bit / 16-bit displacement

2. DATA TRANSFER INSTRUCTIONS

Definition (slide): Instructions that are used to **transfer data/address into registers, memory locations and I/O ports.** Generally involve **two operands — source and destination — of the same size**. The source can be a register, a memory location or immediate data; the destination is a register or a memory location. **The size should be either a byte or a word.** An **8-bit data can only be moved to an 8-bit register/memory** and a **16-bit data to a 16-bit register/memory**.

Mnemonics: MOV, XCHG, PUSH, POP, IN, OUT

Instruction	Operation
`MOV reg2, reg1`	(reg2) ← (reg1)
`MOV mem, reg1`	(mem) ← (reg1)
`MOV reg2, mem`	(reg2) ← (mem)
`MOV reg, data`	(reg) ← data
`MOV mem, data`	(mem) ← data
`XCHG reg2, reg1`	(reg2) ↔ (reg1)
`XCHG mem, reg1`	(mem) ↔ (reg1)
`PUSH reg16`	(SP) ← (SP) – 2 ; MA_S = (SS) × 16_{10} + SP ; (MA_S ; MA_S + 1) ← (reg16)
`PUSH mem`	(SP) ← (SP) – 2 ; MA_S = (SS) × 16_{10} + SP ; (MA_S ; MA_S + 1) ← (mem)
`POP reg16`	MA_S = (SS) × 16_{10} + SP ; (reg16) ← (MA_S ; MA_S + 1) ; (SP) ← (SP) + 2
`POP mem`	MA_S = (SS) × 16_{10} + SP ; (mem) ← (MA_S ; MA_S + 1) ; (SP) ← (SP) + 2
`IN AL, [DX]`	PORTaddr = (DX) ; (AL) ← (PORT)
`IN AX, [DX]`	PORTaddr = (DX) ; (AX) ← (PORT)
`IN AL, addr8`	(AL) ← (addr8)
`IN AX, addr8`	(AX) ← (addr8)
`OUT [DX], AL`	PORTaddr = (DX) ; (PORT) ← (AL)
`OUT [DX], AX`	PORTaddr = (DX) ; (PORT) ← (AX)
`OUT addr8, AL`	(addr8) ← (AL)
`OUT addr8, AX`	(addr8) ← (AX)

Points to remember (very common short questions):

- `MOV` **cannot** transfer **memory → memory**, **segment register → segment register**, or an **immediate → segment register**.
- `XCHG` **swaps** the two operands (no immediate operand allowed).
- **PUSH decrements SP by 2** then stores; **POP loads then increments SP by 2** (stack grows downwards).
- **I/O indirect addressing uses only DX** for the 16-bit port address.

3. ARITHMETIC INSTRUCTIONS

Mnemonics: ADD, ADC, SUB, SBB, INC, DEC, MUL, DIV, IMUL, IDIV, CMP

3.1 ADD / ADC

Instruction	Operation
`ADD reg2, reg1`	(reg2) ← (reg1) + (reg2)
`ADD reg2, mem`	(reg2) ← (reg2) + (mem)
`ADD mem, reg1`	(mem) ← (mem) + (reg1)
`ADD reg, data`	(reg) ← (reg) + data
`ADD mem, data`	(mem) ← (mem) + data
`ADD AL, data8`	(AL) ← (AL) + data8
`ADD AX, data16`	(AX) ← (AX) + data16
`ADC reg2, reg1`	(reg2) ← (reg1) + (reg2) **+ CF**
`ADC reg2, mem`	(reg2) ← (reg2) + (mem) **+ CF**
`ADC mem, reg1`	(mem) ← (mem) + (reg1) **+ CF**
`ADC reg, data`	(reg) ← (reg) + data **+ CF**
`ADC mem, data`	(mem) ← (mem) + data **+ CF**
`ADC AL, data8`	(AL) ← (AL) + data8 **+ CF**
`ADC AX, data16`	(AX) ← (AX) + data16 **+ CF**

ADC = ADD with carry. Used for **multi-byte (multi-word) addition**.

3.2 SUB / SBB

Instruction	Operation
`SUB reg2, reg1`	(reg2) ← (reg1) – (reg2)
`SUB reg2, mem`	(reg2) ← (reg2) – (mem)
`SUB mem, reg1`	(mem) ← (mem) – (reg1)
`SUB reg, data`	(reg) ← (reg) – data
`SUB mem, data`	(mem) ← (mem) – data
`SUB AL, data8`	(AL) ← (AL) – data8
`SUB AX, data16`	(AX) ← (AX) – data16
`SBB reg2, reg1`	(reg2) ← (reg1) – (reg2) **– CF**
`SBB reg2, mem`	(reg2) ← (reg2) – (mem) **– CF**
`SBB mem, reg1`	(mem) ← (mem) – (reg1) **– CF**
`SBB reg, data`	(reg) ← (reg) – data **– CF**
`SBB mem, data`	(mem) ← (mem) – data **– CF**
`SBB AL, data8`	(AL) ← (AL) – data8 **– CF**
`SBB AX, data16`	(AX) ← (AX) – data16 **– CF**

SBB = SUB with borrow. Used for **multi-byte subtraction**.

Note: as written in the slides, `SUB reg2, reg1` is defined as `(reg1) − (reg2)`. Reproduce the slide's convention in your exam answers (the confusion arises because the slides name the destination reg2).

3.3 INC / DEC

Instruction	Operation
`INC reg8`	(reg8) ← (reg8) + 1
`INC reg16`	(reg16) ← (reg16) + 1
`INC mem`	(mem) ← (mem) + 1
`DEC reg8`	(reg8) ← (reg8) – 1
`DEC reg16`	(reg16) ← (reg16) – 1
`DEC mem`	(mem) ← (mem) – 1

INC/DEC do NOT affect the carry flag CF (they do affect ZF, SF, PF, AF, OF).

3.4 MUL / IMUL — unsigned and signed multiplication

Instruction	Byte operation	Word operation
`MUL reg`	**(AX) ← (AL) × (reg8)**	**(DX)(AX) ← (AX) × (reg16)**
`MUL mem`	(AX) ← (AL) × (mem8)	(DX)(AX) ← (AX) × (mem16)
`IMUL reg`	(AX) ← (AL) × (reg8)	(DX)(AX) ← (AX) × (reg16)
`IMUL mem`	(AX) ← (AX) × (mem8) (as printed in the slides)	(DX)(AX) ← (AX) × (mem16)

Key rule (memorise the register pairs):

- **Byte × Byte → 16-bit product in AX** (AL is the implicit multiplicand).
- **Word × Word → 32-bit product in the DX:AX pair** (AX is the implicit multiplicand; **DX holds the high word**).
- **IMUL** = signed multiply — the flags **CF and OF indicate whether the upper half is significant** (useful for exam one-liners).

3.5 DIV / IDIV — unsigned and signed division

Instruction	16 ÷ 8 (word ÷ byte)	32 ÷ 16 (dword ÷ word)
`DIV reg`	**(AL) ← (AX) ÷ (reg8)** → quotient · **(AH) ← (AX) MOD (reg8)** → remainder	**(AX) ← (DX)(AX) ÷ (reg16)** → quotient · **(DX) ← (DX)(AX) MOD (reg16)** → remainder
`DIV mem`	(AL) ← (AX) ÷ (mem8) · (AH) ← (AX) MOD (mem8)	(AX) ← (DX)(AX) ÷ (mem16) · (DX) ← (DX)(AX) MOD (mem16)
`IDIV reg`	(AL) ← (AX) ÷ (reg8) · (AH) ← (AX) MOD (reg8)	(AX) ← (DX)(AX) ÷ (reg16) · (DX) ← (DX)(AX) MOD (reg16)
`IDIV mem`	(AL) ← (AX) ÷ (mem8) · (AH) ← (AX) MOD (mem8)	(AX) ← (DX)(AX) ÷ (mem16) · (DX) ← (DX)(AX) MOD (mem16)

CRITICAL EXAM POINT:

- **Word ÷ Byte:** dividend **AX** → quotient in **AL**, remainder in **AH**.
- **Dword ÷ Word:** dividend **DX:AX** → quotient in **AX**, remainder in **DX**.

- The **remainder always takes the size of the divisor** (i.e. AH for a byte divisor, DX for a word divisor).
- **IDIV** = signed divide, same register conventions.
- If the quotient is too large to fit (or the divisor is 0) → **Type-0 divide-by-zero interrupt** is generated.

3.6 CMP (Compare)

Instruction	Operation
`CMP reg2, reg1`	**Modify flags ← (reg2) – (reg1)**
`CMP reg2, mem`	Modify flags ← (reg2) – (mem)
`CMP mem, reg1`	Modify flags ← (mem) – (reg1)
`CMP reg, data`	Modify flags ← (reg) – data
`CMP mem, data`	Modify flags ← (mem) – data
`CMP AL, data8`	Modify flags ← (AL) – data8
`CMP AX, data16`	Modify flags ← (AX) – data16

Flag table (memorise — it is used for every conditional jump):

Condition	CF	ZF	SF
(reg2) > (reg1)	0	0	0
(reg2) < (reg1)	**1**	0	1
(reg2) = (reg1)	0	**1**	0

CMP modifies flags only — it does NOT change any operand. Always followed by a conditional jump.

4. LOGICAL INSTRUCTIONS (including shifts and rotates)

Mnemonics: AND, OR, XOR, NOT, TEST, SHR, SHL/SAL, RCR, RCL, ROR, ROL

4.1 AND

Instruction	Operation
`AND reg2, reg1`	(reg2) ← (reg2) & (reg1)
`AND reg2, mem`	(reg2) ← (reg2) & (mem)
`AND mem, reg1`	(mem) ← (mem) & (reg1)
`AND reg, data`	(reg) ← (reg) & data
`AND mem, data`	(mem) ← (mem) & data
`AND AL, data8`	(AL) ← (AL) & data8
`AND AX, data16`	(AX) ← (AX) & data16

Uses: masking/clearing bits, checking whether a bit is 0 or 1.

4.2 OR

Instruction	Operation
`OR reg2, reg1`	(reg2) ← (reg2) | (reg1)
`OR reg2, mem`	(reg2) ← (reg2) | (mem)
`OR mem, reg1`	(mem) ← (mem) | (reg1)
`OR reg, data`	(reg) ← (reg) | data
`OR mem, data`	(mem) ← (mem) | data
`OR AL, data8`	(AL) ← (AL) | data8
`OR AX, data16`	(AX) ← (AX) | data16

Use: setting bits.

4.3 XOR

Instruction	Operation
`XOR reg2, reg1`	(reg2) ← (reg2) ^ (reg1)
`XOR reg2, mem`	(reg2) ← (reg2) ^ (mem)
`XOR mem, reg1`	(mem) ← (mem) ^ (reg1)
`XOR reg, data`	(reg) ← (reg) ^ data
`XOR mem, data`	(mem) ← (mem) ^ data
`XOR AL, data8`	(AL) ← (AL) ^ data8
`XOR AX, data16`	(AX) ← (AX) ^ data16

Uses: toggling/complementing selected bits; **`XOR reg, reg` clears a register** (shortcut).

4.4 TEST

Instruction	Operation
`TEST reg2, reg1`	**Modify flags ← (reg2) & (reg1)** (operands unchanged)
`TEST reg2, mem`	Modify flags ← (reg2) & (mem)
`TEST mem, reg1`	Modify flags ← (mem) & (reg1)
`TEST reg, data`	Modify flags ← (reg) & data
`TEST mem, data`	Modify flags ← (mem) & data
`TEST AL, data8`	Modify flags ← (AL) & data8
`TEST AX, data16`	Modify flags ← (AX) & data16

TEST vs AND (slide wording): The TEST instruction **works the same as the AND operation, but unlike the AND instruction it does not change the first operand.** So, if we need to **check whether a number in a register is even or odd**, we can do this using the **TEST instruction without changing the original number**.

4.5 SHR — logical shift right

Instruction forms
`SHR reg, 1` or `SHR reg, CL`
`SHR mem, 1` or `SHR mem, CL`

Operation (slide formula):

```
CF ← B_LSD ; B_n ← B_n+1 ; B_MSD ← 0
```

The **MSB is filled with 0**, the **LSB shifts into CF**, and every bit moves one position right. (The slides show the bit chain B7 … B0 → CF for reg8/mem8 and B15 … B0 → CF for reg16/mem16.)

4.6 SHL / SAL — logical shift left (arithmetic shift left)

Instruction forms
`SHL reg, 1` or `SHL reg, CL` (same for SAL)
`SHL mem, 1` or `SHL mem, CL`

Operation (slide formula):

```
CF ← B_MSD ; B_n+1 ← B_n ; B_LSD ← 0
```

The **LSB is filled with 0**, the **MSB shifts into CF**, and every bit moves one position left.

Doubling trick: SHL by 1 = multiply by 2; SHR by 1 = divide by 2 (unsigned).

Count rule: the count must be either **`1`** (immediate) or **`CL`** — no other register is allowed.

4.7 RCR — rotate through carry right

Instruction forms
`RCR reg, 1` or `RCR reg, CL`
`RCR mem, 1` or `RCR mem, CL`

Operation (slide formula):

```
B_n ← B_n−1 ; B_MSD ← CF ; CF ← B_LSD
```

The bits rotate **right through the carry flag**: the **LSB goes into CF**, and **CF goes into the MSB**. Nothing is lost — the carry is part of the rotation loop.

4.8 RCL — rotate through carry left

Complement of RCR: the bits rotate **left through CF** — **MSB → CF**, **CF → LSB**.

4.9 ROL — rotate left

Instruction forms
`ROL reg, 1` or `ROL reg, CL`
`ROL mem, 1` or `ROL mem, CL`

Operation (slide formula):

```
B_n−1 ← B_n ; CF ← B_MSD ; B_LSD ← B_MSD
```

The **MSB is copied to the LSB and also to CF** — a plain circular rotation (no carry in the loop).

4.10 ROR — rotate right

Complement of ROL: **LSB → MSB** and **LSB → CF**.

Shift/rotate quick table (write this in the exam):

Instruction	Direction	MSB/LSB filled with	CF gets
SHR	right	0 into MSB	old LSB
SHL/SAL	left	0 into LSB	old MSB
RCR	right	**CF** into MSB	old LSB
RCL	left	**CF** into LSB	old MSB
ROR	right	old LSB into MSB	old LSB
ROL	left	old MSB into LSB	old MSB

5. STRING MANIPULATION INSTRUCTIONS

Facts to quote first:

- **String:** a **sequence of bytes or words**.
- The 8086 instruction set includes instructions for string **movement, comparison, scan, load and store**.
- The **REP instruction prefix** is used to **repeat execution** of string instructions.
- String instructions **end with S, SB or SW** — **S** = String, **SB** = String Byte, **SW** = String Word.
- The **offset or effective address of the source operand is stored in the SI register** and that of the **destination operand in the DI register**.
- **Depending on the status of DF, the SI and DI registers are automatically updated:**
  - **DF = 0 ⇒ SI and DI are incremented** by 1 for byte and 2 for word.
  - **DF = 1 ⇒ SI and DI are decremented** by 1 for byte and 2 for word.

Mnemonics: REP, MOVS, CMPS, SCAS, LODS, STOS

5.1 REP prefixes

Prefix	Meaning
REP	Repeat while **CX ≠ 0**, and (CX) ← (CX) – 1
REPZ / REPE	While **CX ≠ 0 and ZF = 1**, repeat the string instruction and (CX) ← (CX) – 1 (Repeat CMPS or SCAS until ZF = 0 — result should be zero for condition true)
REPNZ / REPNE	While **CX ≠ 0 and ZF = 0**, repeat the string instruction and (CX) ← (CX) – 1 (Repeat CMPS or SCAS until ZF = 1 — result should not be zero for condition true)

Slide note: Always **REPZ** can be used in association with the string-related operations. **Example: `REPZ MOVSB`.**

5.2 MOVS / MOVSB / MOVSW — move string

```
MA  = (DS) × 16₁₀ + (SI)          ; source
MAE = (ES) × 16₁₀ + (DI)          ; destination
(MAE) ← (MA)                       ; for MOVSB
(MAE ; MAE + 1) ← (MA ; MA + 1)    ; for MOVSW

If DF = 0 → (DI) ← (DI) + 1 ; (SI) ← (SI) + 1     [byte]
            (DI) ← (DI) + 2 ; (SI) ← (SI) + 2     [word]
If DF = 1 → (DI) ← (DI) − 1 ; (SI) ← (SI) − 1     [byte]
            (DI) ← (DI) − 2 ; (SI) ← (SI) − 2     [word]
```

5.3 CMPS / CMPSB / CMPSW — compare string

Compare two string bytes or string words.

```
MA  = (DS) × 16₁₀ + (SI)
MAE = (ES) × 16₁₀ + (DI)
Modify flags ← (MA) − (MAE)

If (MA) > (MAE)  → CF = 0 ; ZF = 0 ; SF = 0
If (MA) < (MAE)  → CF = 1 ; ZF = 0 ; SF = 1
If (MA) = (MAE)  → CF = 0 ; ZF = 1 ; SF = 0

For byte operation: DF = 0 → (SI),(DI) + 1 ; DF = 1 → (SI),(DI) − 1
For word operation: DF = 0 → (SI),(DI) + 2 ; DF = 1 → (SI),(DI) − 2
```

5.4 SCAS / SCASB / SCASW — scan string

Scan (compare) a string byte or word with the accumulator.

```
MAE = (ES) × 16₁₀ + (DI)
Modify flags ← (AL) − (MAE)          [SCASB]
Modify flags ← (AX) − (MAE ; MAE+1)  [SCASW]

If (AL) > (MAE)  → CF = 0 ; ZF = 0 ; SF = 0
If (AL) < (MAE)  → CF = 1 ; ZF = 0 ; SF = 1
If (AL) = (MAE)  → CF = 0 ; ZF = 1 ; SF = 0

If DF = 0 → (DI) ← (DI) + 1 (or +2 for word)
If DF = 1 → (DI) ← (DI) − 1 (or −2 for word)
```

5.5 LODS / LODSB / LODSW — load string

Load string byte into AL or string word into AX.

```
MA = (DS) × 16₁₀ + (SI)
(AL) ← (MA)              [LODSB]
(AX) ← (MA ; MA + 1)     [LODSW]

If DF = 0 → (SI) ← (SI) + 1 (or +2 for word)
If DF = 1 → (SI) ← (SI) − 1 (or −2 for word)
```

5.6 STOS / STOSB / STOSW — store string

Store byte from AL or word from AX into string.

```
MAE = (ES) × 16₁₀ + (DI)
(MAE) ← (AL)                [STOSB]
(MAE ; MAE + 1) ← (AX)      [STOSW]

If DF = 0 → (DI) ← (DI) + 1 (or +2 for word)
If DF = 1 → (DI) ← (DI) − 1 (or −2 for word)
```

Memory table of string instructions:

Instruction	Operands involved	Segment used	Which registers auto-update
MOVS	source SI → destination DI	DS → ES	SI and DI
CMPS	source SI vs destination DI	DS vs ES	SI and DI
SCAS	**AL/AX** vs destination DI	ES	DI only
LODS	source SI → **AL/AX**	DS	SI only
STOS	**AL/AX** → destination DI	ES	DI only

6. PROCESSOR CONTROL INSTRUCTIONS

Mnemonic	Explanation
STC	**Set** carry: CF ← 1
CLC	**Clear** carry: CF ← 0
CMC	**Complement** carry: CF ← CF′
STD	**Set** direction flag: DF ← 1
CLD	**Clear** direction flag: DF ← 0
STI	**Set** interrupt enable flag: IF ← 1
CLI	**Clear** interrupt enable flag: IF ← 0
NOP	**No operation**
HLT	**Halt** after interrupt is set
WAIT	**Wait** for TEST pin active
ESC opcode mem/reg	Used to **pass instruction to a coprocessor** which shares the address and data bus with the 8086
LOCK	**Lock the bus** during the next instruction (prevents other bus masters taking the bus)

Note: STI/CLI affect **maskable** interrupts only — the **NMI can never be masked** in software.

7. CONTROL TRANSFER INSTRUCTIONS

Facts to quote first:

- They **transfer the control to a specific destination or target instruction.**
- They **do NOT affect flags.**

7.1 Unconditional transfers

Mnemonic	Explanation
`CALL reg/mem/disp16`	**Call subroutine**
`RET`	**Return from subroutine**
`JMP reg/mem/disp8/disp16`	**Unconditional jump**

7.2 Conditional branch instructions

- They **check flags**.
- If the conditions are **true**, the program control is transferred to the **new memory location in the same segment by modifying the content of IP**.

SIGNED conditional branch instructions:

Name	Alternate name
`JE disp8`	Jump if equal
`JZ disp8`	Jump if result is 0
`JNE disp8`	Jump if not equal
`JNZ disp8`	Jump if not zero
`JG disp8`	Jump if greater
`JNLE disp8`	Jump if not less or equal
`JGE disp8`	Jump if greater than or equal
`JNL disp8`	Jump if not less
`JL disp8`	Jump if less than
`JNGE disp8`	Jump if not greater than or equal
`JLE disp8`	Jump if less than or equal
`JNG disp8`	Jump if not greater

UNSIGNED conditional branch instructions:

Name	Alternate name
`JE disp8`	Jump if equal
`JZ disp8`	Jump if result is 0
`JNE disp8`	Jump if not equal
`JNZ disp8`	Jump if not zero
`JA disp8`	Jump if above
`JNBE disp8`	Jump if not below or equal
`JAE disp8`	Jump if above or equal
`JNB disp8`	Jump if not below
`JB disp8`	Jump if below
`JNAE disp8`	Jump if not above or equal
`JBE disp8`	Jump if below or equal
`JNA disp8`	Jump if not above

Slide note: Before these instructions, a **comparison instruction is to be used** for comparing two operands (i.e. use `CMP` first).
Signed vs unsigned trick: Above/Below (A/B) are for **unsigned** data; **Greater/Less** (G/L) are for **signed** data.

Conditional branch instructions affecting INDIVIDUAL FLAGS:

Mnemonic	Explanation
`JC disp8`	Jump if **CF = 1**
`JNC disp8`	Jump if **CF = 0**
`JP disp8`	Jump if **PF = 1**
`JNP disp8`	Jump if **PF = 0**
`JO disp8`	Jump if **OF = 1**
`JNO disp8`	Jump if **OF = 0**
`JS disp8`	Jump if **SF = 1**
`JNS disp8`	Jump if **SF = 0**
`JZ disp8`	Jump if result is zero, i.e. **ZF = 1**
`JNZ disp8`	Jump if result is not zero, i.e. **ZF = 0**

7.3 Iteration control (loop) instructions

These instructions are used to execute the given instructions for a number of times.

Instruction	Meaning
`LOOP`	Used to **loop a group of instructions until the condition satisfies, i.e. CX = 0**
`LOOPE / LOOPZ`	Loop a group of instructions till it satisfies **ZF = 1 & CX = 0**
`LOOPNE / LOOPNZ`	Loop a group of instructions till it satisfies **ZF = 0 & CX = 0**
`JCXZ`	Used to **jump to the provided address if CX = 0**

Example (from the slides):

```
MOV CX, 06H
label: ADD AL, BL
       LOOP label        ; decrements CX (without affecting flags) and loops while CX ≠ 0
```

8. ASSEMBLER DIRECTIVES AND OPERATORS

8.1 What is an assembler directive?

- **Instructions to the assembler** regarding the program being executed.
- They **control the generation of machine codes and organisation of the program**; but **no machine codes are generated for assembler directives.**
- Also called **'pseudo instructions'.**

Used to:

- specify the **start and end of a program**
- attach **value to variables**
- allocate **storage locations** to input/output data
- define **start and end of segments, procedures, macros, etc.**

List of directives used in the slides: DB, DW, SEGMENT, ENDS, ASSUME, ORG, END, EVEN, EQU, PROC, FAR, NEAR, ENDP, SHORT, MACRO, ENDM

8.2 DB — Define Byte

- Defines a **byte type (8-bit) variable**.
- **Reserves specific amount of memory locations to each variable.**
- **Range: 00H – FFH** for unsigned value; **00H – 7FH** for positive value and **80H – FFH** for negative value.
- **General form:** `variable DB value/values`
- **Example:**

```
LIST DB 7FH, 42H, 35H
```

→ Three consecutive memory locations are reserved for the variable LIST, and each data specified in the instruction is stored as the initial value in the reserved memory location.

8.3 DW — Define Word

- Defines a **word type (16-bit) variable**.
- **Reserves two consecutive memory locations to each variable.**
- **Range: 0000H – FFFFH** for unsigned value; **0000H – 7FFFH** for positive value and **8000H – FFFFH** for negative value.
- **General form:** `variable DW value/values`
- **Example:**

```
ALIST DW 6512H, 0F251H, 0CDE2H
```

→ Six consecutive memory locations are reserved for the variable ALIST, each 16-bit data stored in two consecutive memory locations.

8.4 The remaining directives

Directive	Meaning
SEGMENT	Used to indicate the **beginning** of a code/data/stack segment. General form: `Segnam SEGMENT … Segnam ENDS` (the user-defined name of the segment appears before SEGMENT and before ENDS).
ENDS	Used to indicate the **end** of a code/data/stack segment.
ASSUME	**Informs the assembler the name of the program/data segment that should be used for a specific segment register.** General form: `ASSUME segreg : segnam, …, segreg : segnam` — e.g. `ASSUME CS: ACODE, DS: ADATA` tells the compiler that the instructions of the program are stored in segment **ACODE** and data are stored in segment **ADATA**.
ORG	**Origin** — used to **assign the starting address (Effective address)** for a program/data segment. `ORG 1000H` informs the assembler that the statements following ORG 1000H should be stored in memory starting with effective address 1000H.
END	Used to **terminate a program**; **statements after END will be ignored.**
EVEN	Informs the assembler to **store a program/data segment starting from an even address.**
EQU	**Equate** — used to **attach a value to a variable.** `LOOP EQU 10FEH` → the value of variable LOOP is 10FEH.
PROC	Indicates the **beginning of a procedure.** `procname PROC [NEAR/FAR] … RET … procname ENDP`
ENDP	**End of procedure.** PROC and ENDP **bracket** a procedure.
FAR	**Intersegment call** (the procedure is in a different code segment).
NEAR	**Intrasegment call** (the procedure is in the same code segment).
SHORT	**Reserves one memory location for an 8-bit signed displacement** in jump instructions. `JMP SHORT AHEAD` → the directive reserves one memory location for the 8-bit displacement named AHEAD.
MACRO	Indicates the **beginning of a macro.** `macroname MACRO [Arg1, Arg2 …] … ENDM`
ENDM	**End of a macro.**

Worked directive examples (from the slides):

```
_SDATA SEGMENT
       ORG 1200H
       A DB 4CH
       EVEN
       B DW 1052H
_SDATA ENDS
```

→ In this data segment, the **effective address of memory location assigned to A will be 1200H** and that of **B will be 1202H and 1203H**.

```
ADD64 PROC NEAR          ; the assembler will code the CALL and RET
   …                     ; instructions involved in this procedure
   RET                   ; as NEAR call and return
ADD64 ENDP

CONVERT PROC FAR         ; the assembler will code CALL and RET
   …                     ; as FAR call and return
   RET
CONVERT ENDP
```

8.5 Assembler operators: LENGTH, OFFSET, LEA

Operator	Meaning
LENGTH	An **operator which tells the assembler to determine the number of elements** in some named data item, such as a string or an array. When the assembler reads `MOV CX, LENGTH STRING1`, it determines the number of elements in STRING1 and loads it into CX. If the string was declared as a **byte string, LENGTH produces the number of bytes**; if declared as a **word string, LENGTH produces the number of words**. Byte length of a label: used to refer to the length of a data array or a string — `MOV CX, LENGTH ARRAY`.
OFFSET	**Offset of a label:** when the assembler comes across the OFFSET operator along with a label, it **first computes the 16-bit offset address of that particular label** and **replaces the string 'OFFSET LABEL' by the computed offset address.** Example: `MOV SI, OFFSET LIST`
LEA	**Load Effective Address — loads the address of a variable.** Example: `TEST DB 23H, 40H, 44H` then `LEA AX, TEST`

9. PROCEDURES, CALL/RET, MACROS AND MODULAR PROGRAMMING

9.1 Procedure

Definition: A procedure is a **group of instructions that usually performs one task**. It is a **reusable section of a software program which is stored in memory once but can be used as often as necessary.**

Two types:

- **NEAR procedure:** a procedure is known as a **NEAR procedure if it is written (defined) in the same code segment which is calling that procedure. Only the Instruction Pointer (IP) register content will be changed.**
- **FAR procedure:** a procedure is known as a **FAR procedure if it is written (defined) in a different code segment than the calling segment.** In this case **both the IP and the Code Segment (CS) register contents will be changed.**

Directives used for a procedure:

- **PROC directive:** used to identify the **start** of a procedure; follows a name given to the procedure; after that the term **FAR or NEAR** specifies the type of the procedure.
- **ENDP directive:** used along with the name of the procedure to indicate the **end** of a procedure to the assembler.
- The **PROC and ENDP directives are used to bracket a procedure.**

9.2 CALL and RET instructions

CALL instruction: used to transfer execution to a procedure. It performs **two operations**:

1. First it **stores the address of the instruction after the CALL instruction on the stack.**
2. Second it **changes the content of the IP register** in case of a Near call **and changes the content of the IP and CS registers** in case of a FAR call.

Two types of calls: Near Call (Intrasegment call) and **Far Call (Intersegment call)**.

Operation for Near Call: When the 8086 executes a near CALL instruction, it **decrements the stack pointer by 2** and **copies the IP register contents onto the stack**. Then it **copies the address of the first instruction of the called procedure** (into IP).

Operation of FAR CALL: When the 8086 executes a far call, it:

1. **decrements the stack pointer by 2** and **copies the contents of the CS register to the stack**;
2. **decrements the stack pointer by 2 again** and **copies the content of the IP register to the stack**;
3. finally **loads the CS register with the base address of the segment having the procedure** and **IP with the address of the first instruction in the procedure**.

Stack order — FAR CALL pushes CS first, then IP (so RET/IRET can pop IP first). NEAR CALL pushes only IP.

9.3 Macro

- **MACRO Directive:** indicates the **beginning of a macro**; **ENDM:** end of a macro.
- **General form:**

```
macroname MACRO [Arg1, Arg2 ...]
    … program statements in the macro …
ENDM
```

9.4 Procedure vs Macro (COMPARISON TABLE — high-yield)

Procedure	Macro
Procedures are used for **large** group of instructions to be repeated	Macros are used for **small** group of instructions to be repeated
Object code is generated **only once** in memory	Object code is generated **every time the macro is called**
CALL & RET instructions are used to call procedure and return from procedure	Macro can be called **just by writing its name**
Length of the object file is **less**	Object file becomes **lengthy**
Directives **PROC & ENDP** are used for defining procedure	Directives **MACRO and ENDM** are used for defining MACRO
More time is required for its execution	**Less time** is required for its execution
Procedure can be defined as: `Procedure_name PROC … Procedure_name ENDP`	Macro can be defined as: `MACRO-name MACRO [ARGUMENT, … ARGUMENT N] … ENDM`
For example: `Addition PROC near` … `Addition ENDP`	For example: `Display MACRO msg` … `ENDM`

9.5 Modular programming

Definition: Modular programming is a **software design technique that emphasises separating the functionality of a program into independent, interchangeable modules**, such that **each contains everything necessary to execute only one aspect of the desired functionality.**

- A **module interface** expresses the elements that are **provided and required** by the module. The elements defined in the interface are **detectable by other modules**.
- The **implementation** contains the working code that corresponds to the elements declared in the interface.
- Modular programming is **closely related to structured programming and object-oriented programming**, all having the **same goal of facilitating construction of large software programs and systems by decomposition into smaller pieces**, and all originating around the 1960s.
- While historical usage of these terms has been inconsistent, **"modular programming" now refers to high-level decomposition of the code of an entire program into pieces**; **structured programming** to the low-level code use of **structured control flow**; and **object-oriented programming** to the **data use of objects** (a kind of data structure).

10. EXAMPLE PROGRAMS

10.1 Procedure example program (with data + stack + code segments)

```
ASSUME CS:CODE, DS:DATA, SS:STACK_SEG

DATA SEGMENT
  NUM1    DB 50H
  NUM2    DB 20H
  ADD_RES DB ?
  SUB_RES DB ?
DATA ENDS

STACK_SEG SEGMENT
  DW 40 DUP(0)          ; stack of 40 words, all initialized to zero
  TOS LABEL WORD
STACK_SEG ENDS

CODE SEGMENT
START: MOV AX, DATA          ; initialize data segment
       MOV DS, AX
       MOV AX, STACK_SEG     ; initialize stack segment
       MOV SS, AX
       MOV SP, OFFSET TOS    ; initialize stack pointer to TOS
       CALL ADDITION
       CALL SUBTRACTION
       MOV AH, 4CH           ; return to DOS
       INT 21H

ADDITION PROC NEAR
       MOV AL, NUM1
       MOV BL, NUM2
       ADD AL, BL
       MOV ADD_RES, AL
       RET
ADDITION ENDP

SUBTRACTION PROC
       MOV AL, NUM1
       MOV BL, NUM2
       SUB AL, BL
       MOV SUB_RES, AL
       RET
SUBTRACTION ENDP

CODE ENDS
END START
```

Points to write as comments in the exam:

- `MOV AX, DATA` followed by `MOV DS, AX` — a **segment register cannot be loaded with an immediate**, so the segment base is first loaded into a general-purpose register (AX) and then moved to DS/SS.
- `MOV SP, OFFSET TOS` initialises the **stack pointer** to the top of stack.
- `MOV AH, 4CH` + `INT 21H` = **DOS terminate program**.
- `PROC NEAR` … `RET` … `ENDP` bracket each procedure.

10.2 Macro example program

```
ASSUME CS:CODE, DS:DATA

DATA SEGMENT
  NUM1 DW 1000H
  NUM2 DW 2000H
  RES  DW ?
DATA ENDS

CODE SEGMENT
ADDITION MACRO NO1, NO2, RESULT
         MOV AX, NO1
         MOV BX, NO2
         ADD AX, BX
         MOV RESULT, AX
         ENDM

START: MOV AX, DATA            ; initialize data segment
       MOV DS, AX
       ADDITION NUM1, NUM2, RES    ; macro call — just write its name
       MOV AH, 4CH
       INT 21H
CODE ENDS
END START
```

10.3 Divide-by-zero demonstration (Emu8086 — Type-0 interrupt)

```
MOV AL, 05H
MOV DL, 00H
DIV DL        ; → divide by zero → Type 0 interrupt
HLT
```

10.4 Overflow demonstration (Type-4 interrupt)

A signed arithmetic overflow (the slides show an `INT32` overflow exception in a high-level program) is handled through the **Type-4 Overflow interrupt** path — i.e. an arithmetic operation whose signed result does not fit the destination raises the overflow error condition.

11. MOST LIKELY EXAM QUESTIONS (INSTRUCTION SET)

Question	Answer skeleton
Explain the instruction set of 8086 / classify the instructions of 8086.	The **6 groups**: data transfer, arithmetic, logical, string manipulation, processor control, control transfer — with 2–3 example mnemonics each, then elaborate any one group.
Explain the data transfer instructions with examples (MOV, XCHG, PUSH, POP, IN, OUT).	The operation table of §2; mention MOV restrictions, PUSH/POP SP changes, IN/OUT via DX or addr8.
Explain MUL and DIV instructions with examples.	Byte: AL × operands → AX; Word: AX × operands → DX:AX. DIV: AX ÷ r8 → AL quotient, AH remainder; DX:AX ÷ r16 → AX quotient, DX remainder; mention Type-0 on divide overflow.

Question	Answer skeleton
Differentiate AND and TEST.	Both perform the AND operation, but **TEST does not change the first operand** — used to check even/odd without altering the number.
Explain the shift and rotate instructions of 8086.	SHR, SHL/SAL, RCR, RCL, ROR, ROL with the CF/bit-flow table of §4; count can be 1 or `CL` only.
Explain the string instructions of 8086.	MOVS, CMPS, SCAS, LODS, STOS + REP/REPZ/REPNZ; source DS:SI, destination ES:DI; DF controls auto increment/decrement (1 for byte, 2 for word).
What does REP do? Difference between REPZ and REPNZ?	REP repeats while CX ≠ 0; REPZ/REPE repeats while CX ≠ 0 **and ZF = 1**; REPNZ/REPNE repeats while CX ≠ 0 **and ZF = 0**.
Explain the control transfer / branch instructions of 8086.	Unconditional (CALL, RET, JMP); signed conditional (JG/JGE/JL/JLE …); unsigned conditional (JA/JAE/JB/JBE …); flag-based (JC/JNC/JP/JO/JS/JZ …); note CMP is used first and control transfers modify IP within the same segment.
Explain the loop instructions with an example.	LOOP till CX = 0; LOOPE/LOOPZ till ZF = 1 & CX ≠ 0; LOOPNE/LOOPNZ till ZF = 0 & CX ≠ 0; JCXZ jumps if CX = 0 — with the `MOV CX,06H / label: ADD AL,BL / LOOP label` example.
What are assembler directives? Explain DB, DW, SEGMENT, ASSUME, ORG, EQU, PROC, ENDP, MACRO, ENDM.	Directives definition (pseudo instructions, no machine code generated) + the meaning table + the DB/DW range tables of §8.
Differentiate DB and DW.	DB = 8-bit variable, 1 location, 00H–FFH; DW = 16-bit variable, 2 consecutive locations, 0000H–FFFFH.
Explain LENGTH, OFFSET and LEA with examples.	LENGTH = number of elements of an array/string; OFFSET = 16-bit offset address of a label; LEA = load effective address of a variable.
Differentiate procedures and macros.	The full 7-row comparison table of §9.4.
Difference between NEAR and FAR procedure/call.	NEAR: same code segment, **only IP** pushed/changed. FAR: different segment, **CS then IP** pushed, both changed.
Write an ALP to add/subtract two numbers using a procedure.	The program of §10.1.
Write an ALP using a macro.	The program of §10.2.
What is modular programming?	Definition + interface vs implementation + relation to structured/OOP programming.

12. LAST-MINUTE CRAM SHEET (INSTRUCTION SET)

The 6 groups: D-A-L-S-P-C = Data transfer, Arithmetic, Logical, String, Processor control, Control transfer.

Register rules to never forget:

Situation	Result register
Byte MUL	**AX** ← AL × reg8/mem8
Word MUL	**DX:AX** ← AX × reg16/mem16
Word ÷ Byte	**AL = quotient, AH = remainder**
Dword ÷ Word	**AX = quotient, DX = remainder**
Shift/rotate count	**`1` or `CL` only**
Loop counter	**CX**
String source / destination	**DS:SI / ES:DI**
I/O indirect port address	**DX only**

ADC/SBB: ADD/SUB **with carry/borrow** → used for multi-byte arithmetic. **INC/DEC do not affect CF. CMP and TEST modify flags only** — no operand change. **Uses:** AND = mask/clear bits · OR = set bits · XOR = toggle bits (`XOR reg,reg` clears) · TEST = check even/odd without modifying · SHL = ×2 · SHR(unsigned) = ÷2. **DF in strings:** DF = 0 → increment SI/DI; DF = 1 → decrement SI/DI (by 1 for byte, 2 for word). **Conditional jumps:** A/B family = **unsigned**; G/L family = **signed**; JZ/JNZ, JC/JNC, JP/JNP, JO/JNO, JS/JNS = individual flags. Use **CMP first**. **Assembler directives generate NO machine code** — they are **pseudo instructions**. **PROC/ENDP** bracket a procedure (**CALL & RET**, object code once) vs **MACRO/ENDM** bracket a macro (**called by name**, code expanded every time, faster but lengthier object file). **FAR call pushes CS then IP; NEAR call pushes only IP.**

Cross-check with Part 1 (Architecture) and Part 2 (Addressing Modes) — together the three files cover the complete Module 2 syllabus.